Operating System MCQs for Welldev Interview Prep (100 Questions with Explanations)

Section 1: Basics of OS, Process Management (Q1–Q20)

Q1. What is an Operating System? A) Application software B) System software that manages hardware and software resources C) A type of hardware D) A programming language

Answer: B — The OS acts as an intermediary between users/applications and computer hardware, managing resources like CPU, memory, and I/O devices.

Q2. Which of the following is NOT a function of an Operating System? A) Memory management B) Process management C) Compiling source code D) File management

Answer: C — Compiling is done by a compiler (application-level tool), not the OS itself. The OS provides process, memory, file, and device management.

Q3. A process in memory is described by which data structure? A) Process Control Block (PCB) B) Thread Control Block C) Inode D) File Allocation Table

Answer: A — The PCB stores process state, program counter, CPU registers, memory info, and I/O status information for each process.

Q4. What are the typical states of a process? A) New, Ready, Running, Waiting, Terminated B) Start, Stop, Pause C) Open, Close D) Active, Inactive

Answer: A — These are the five standard states in the process lifecycle model.

Q5. What is context switching? A) Switching between two files B) Saving the state of a process and loading the state of another C) Switching between two users D) Changing the OS

Answer: B — Context switching lets the CPU move from executing one process/thread to another, saving/restoring register and PCB data, which incurs overhead.

Q6. Which scheduler decides which process moves from ready to running? A) Long-term scheduler B) Short-term scheduler (CPU scheduler) C) Medium-term scheduler D) I/O scheduler

Answer: B — The short-term scheduler runs frequently and selects a process from the ready queue for execution.

Q7. What is the main purpose of the long-term scheduler? A) Selects which process to run next on CPU B) Controls the degree of multiprogramming by admitting jobs into memory C) Swaps processes in/out of memory D) Handles disk I/O

Answer: B — The long-term (job) scheduler decides which processes are admitted into the ready queue, controlling multiprogramming level.

Q8. What is a zombie process? A) A process that has finished execution but still has an entry in the process table B) A process stuck in an infinite loop C) A process with no parent D) A corrupted process

Answer: A — A zombie process has completed execution, but its exit status hasn't been read by the parent, so its PCB entry remains until the parent calls wait().

Q9. What is an orphan process? A) A process whose parent has terminated before it B) A process without any memory allocated C) A process running without CPU time D) A duplicate process

Answer: A — When a parent terminates before its child, the child becomes an orphan and is typically adopted by the init/systemd process (PID 1).

Q10. Difference between process and thread — which is TRUE? A) Threads share the same address space; processes do not B) Processes are lighter than threads C) Threads cannot share memory D) A process can have only one thread

Answer: A — Threads within the same process share code, data, and OS resources (address space), making context switching between threads cheaper than between processes.

Q11. What is multithreading? A) Running multiple processes on multiple CPUs B) Executing multiple threads within a single process concurrently C) Running multiple operating systems D) Multiple users using one program

Answer: B — Multithreading allows a single process to have multiple execution paths (threads), enabling concurrency within an application.

Q12. Which is an advantage of multithreading? A) Increased memory usage per thread B) Responsiveness, resource sharing, and better CPU utilization C) Slower context switches D) More complex than multiprocessing always

Answer: B — Because threads share resources, they're cheaper to create/switch than processes, improving responsiveness and utilization.

Q13. What is the difference between user-level threads and kernel-level threads? A) User threads are managed by the OS kernel; kernel threads by libraries B) User threads are

managed by a thread library without kernel awareness; kernel threads are managed directly by the OS C) There is no difference D) Kernel threads are always faster to create

Answer: B — User-level threads are faster to create/manage but the kernel is unaware of them (blocking one blocks all); kernel threads are scheduled by the OS itself.

Q14. What is a fork() system call used for (in Unix/Linux)? A) To terminate a process B) To create a new (child) process C) To create a new thread D) To allocate memory

Answer: B — fork() creates a new process by duplicating the calling (parent) process; the child gets a copy of the parent's address space.

Q15. What does exec() do after a fork()? A) Terminates the child B) Replaces the process image with a new program C) Creates another child D) Pauses the process

Answer: B — exec() loads a new program into the current process's memory space, replacing the old program — commonly used after fork() to run a different program in the child.

Q16. What is an interrupt? A) A signal that causes the CPU to stop its current task and handle an event B) A type of process C) A memory management technique D) A scheduling algorithm

Answer: A — Interrupts (hardware or software) pause normal execution flow so the CPU can respond to events like I/O completion or errors.

Q17. What is a system call? A) A call between two applications B) An interface for a process to request a service from the OS kernel C) A function call between threads D) A hardware interrupt only

Answer: B — System calls (e.g., read, write, fork) let user programs request kernel services like file I/O, process control, and communication.

Q18. Which mode does a system call typically require? A) User mode only B) Kernel mode (privileged mode) C) Safe mode D) Guest mode

Answer: B — System calls trigger a mode switch from user mode to kernel mode so the OS can perform privileged operations safely.

Q19. What is IPC (Inter-Process Communication)? A) A method for processes to communicate/synchronize with each other B) A CPU scheduling algorithm C) A memory allocation strategy D) A file system type

Answer: A — IPC mechanisms (pipes, message queues, shared memory, sockets, semaphores) allow independent processes to exchange data and coordinate.

Q20. Which IPC mechanism is fastest generally? A) Message passing B) Shared memory C) Pipes D) Sockets

Answer: B — Shared memory avoids kernel involvement for each data transfer (once set up), making it faster than message passing, which requires system calls for every message.

Section 2: CPU Scheduling (Q21–Q40)

Q21. Which scheduling algorithm can cause starvation? A) Round Robin B) FCFS C) Priority Scheduling D) All of the above except Round Robin guarantees fairness

Answer: C — In priority scheduling, low-priority processes may never execute if higher-priority processes keep arriving — this is starvation. Round Robin avoids it due to time-sliced fairness.

Q22. What is the solution to starvation in priority scheduling? A) Aging B) Preemption C) Deadlock detection D) Paging

Answer: A — Aging gradually increases the priority of processes that wait a long time, ensuring they eventually run.

Q23. In Round Robin scheduling, what happens if the time quantum is very large? A) It behaves like SJF B) It behaves like FCFS C) It causes deadlock D) It behaves like Priority Scheduling

Answer: B — If the quantum is larger than the longest burst time, each process runs to completion before being preempted, effectively becoming FCFS.

Q24. In Round Robin, if the time quantum is very small, what is the drawback? A) Starvation B) High context-switch overhead C) Deadlock D) No drawback

Answer: B — Too-frequent switching increases the number of context switches, adding CPU overhead and reducing throughput.

Q25. What is SJF (Shortest Job First) scheduling? A) Executes the process with the smallest burst time next B) Executes processes in arrival order C) Executes the longest process first D) Executes based on priority number

Answer: A — SJF selects the ready process with the shortest CPU burst time, minimizing average waiting time (optimal for minimizing average wait, given known burst times).

Q26. What is a major drawback of SJF? A) High average waiting time B) It requires knowing burst times in advance, and can cause starvation of long processes C) It is always preemptive D) It cannot be implemented

Answer: B — Predicting burst time isn't always possible, and long processes may indefinitely wait if short jobs keep arriving (starvation).

Q27. What is SRTF (Shortest Remaining Time First)? A) Non-preemptive version of SJF B) Preemptive version of SJF, where a new shorter job can preempt current running job C) A round robin variant D) A priority variant only

Answer: B — SRTF preempts the current process if a newly arrived process has a shorter remaining burst time than the currently running one.

Q28. What does FCFS stand for and what's its main issue? A) First Come First Serve; suffers from convoy effect B) Fastest CPU First Serve; suffers from deadlock C) First Come First Serve; is always optimal D) Fixed CPU First Serve

Answer: A — FCFS executes processes in arrival order. The convoy effect occurs when a long process holds up many short processes behind it, increasing average wait time.

Q29. Which scheduling algorithm is used in real-time systems requiring strict deadlines? A) FCFS B) Rate Monotonic Scheduling / Earliest Deadline First C) Round Robin only D) SJF only

Answer: B — Real-time scheduling algorithms like Rate Monotonic (RM) or Earliest Deadline First (EDF) explicitly account for task deadlines and periods.

Q30. What is Multilevel Queue Scheduling? A) One single queue with dynamic priority B) Ready queue is divided into separate queues based on process type/priority, each with its own scheduling algorithm C) A single algorithm for all processes D) Scheduling based only on arrival time

Answer: B — Processes are permanently assigned to a queue (e.g., system, interactive, batch) with each queue possibly using a different algorithm, plus inter-queue scheduling.

Q31. How does Multilevel Feedback Queue differ from Multilevel Queue? A) Processes can move between queues based on behavior/aging B) Processes are fixed to one queue permanently C) It only has one queue D) There's no difference

Answer: A — MLFQ allows processes to move between queues (e.g., CPU-bound processes demoted, I/O-bound/interactive processes promoted), offering more flexibility and preventing starvation via aging.

Q32. What metric does "turnaround time" measure? A) Time from submission to completion of a process B) Time a process waits in ready queue C) Time to execute one instruction D) Time between two interrupts

Answer: A — Turnaround time = Completion time − Arrival time; it includes waiting, execution, and I/O time.

Q33. What is "waiting time" in scheduling? A) Total time a process spends in the ready queue waiting for CPU B) Time taken for I/O C) Time to load a process into memory D) Time between context switches

Answer: A — Waiting time = Turnaround time − Burst time, representing time spent waiting rather than executing.

Q34. What is "response time"? A) Time from submission until the first response/output is produced B) Time to terminate a process C) Total execution time D) Same as turnaround time

Answer: A — Especially important in interactive systems, response time measures how quickly a process starts producing output, not necessarily finishing.

Q35. Which scheduling algorithm minimizes average waiting time (assuming all jobs available at once)? A) FCFS B) SJF C) Round Robin D) Priority

Answer: B — SJF is provably optimal for minimizing average waiting time when all burst times are known and jobs arrive together.

Q36. What is preemptive scheduling? A) CPU can be forcibly taken away from a running process B) A process runs to completion once started C) Only used in batch systems D) Same as non-preemptive

Answer: A — In preemptive scheduling, a higher-priority or shorter process can interrupt the currently running process (e.g., SRTF, Round Robin, preemptive priority).

Q37. Which scheduling algorithms are non-preemptive by nature? A) FCFS and (non-preemptive) SJF B) Round Robin C) SRTF D) Preemptive Priority

Answer: A — FCFS never preempts, and basic SJF (unless specified as SRTF) also runs each job to completion once started.

Q38. What is the "convoy effect"? A) Multiple short processes waiting for one long process to release CPU B) Deadlock among processes C) Memory fragmentation D) Two processes accessing shared memory simultaneously

Answer: A — Occurs mainly in FCFS scheduling: a CPU-bound process blocks I/O-bound processes behind it, reducing overall efficiency.

Q39. In priority scheduling, lower number typically means? A) Lower priority B) Higher priority (convention-dependent but common in OS textbooks) C) No priority D) Always equal priority

Answer: B — Many textbook priority schemes treat a smaller number as higher priority — always confirm convention per system since it can vary.

Q40. Which scheduling algorithm is best suited for time-sharing systems? A) FCFS B) Round Robin C) SJF (non-preemptive) D) Batch scheduling

Answer: B — Round Robin's time-quantum-based fairness makes it ideal for time-sharing/interactive systems where responsiveness matters.

Section 3: Process Synchronization & Deadlocks (Q41–Q60)

Q41. What is a critical section? A) A part of code where shared resources are accessed and must not be executed by more than one process at a time B) The main function of a program C) A section of memory that is read-only D) A section reserved for the OS kernel only

Answer: A — The critical section problem deals with ensuring mutual exclusion when multiple processes/threads access shared resources.

Q42. What are the three requirements a solution to the critical section problem must satisfy? A) Mutual exclusion, Progress, Bounded waiting B) Speed, Memory, Cost C) Deadlock, Starvation, Livelock D) Priority, Fairness, Speed

Answer: A — These are the classic requirements: mutual exclusion (only one process in CS at a time), progress (no indefinite postponement), and bounded waiting (limit on how long a process waits).

Q43. What is a semaphore? A) A hardware device B) An integer variable used for signaling and synchronization between processes C) A type of scheduling algorithm D) A memory management unit

Answer: B — A semaphore is accessed via atomic wait() (P) and signal() (V) operations, used for mutual exclusion and coordinating process/thread execution.

Q44. What is the difference between a binary semaphore and a counting semaphore? A) Binary semaphore can only be 0 or 1; counting semaphore can take any non-negative integer value B) Both are the same C) Counting semaphores can only be 0 or 1 D) Binary semaphores allow multiple resource instances

Answer: A — Binary semaphores (mutex-like) control access to a single resource; counting semaphores manage a pool of multiple resource instances.

Q45. What is a mutex? A) A locking mechanism ensuring only one thread accesses a resource at a time, with ownership B) Same as a counting semaphore C) A scheduling algorithm D) A memory allocation technique

Answer: A — A mutex (mutual exclusion lock) is typically owned by the thread that locks it, and only that thread can unlock it — unlike semaphores.

Q46. What is the classic "Producer-Consumer Problem"? A) A scheduling problem B) A synchronization problem where producers generate data into a shared buffer and consumers remove it, requiring coordination to avoid overflow/underflow C) A deadlock avoidance algorithm D) A memory paging technique

Answer: B — Solved using semaphores (empty, full, mutex) to synchronize access to a bounded buffer between producer and consumer processes.

Q47. What is the "Dining Philosophers Problem" used to illustrate? A) CPU scheduling B) Deadlock and synchronization issues with shared resources C) Memory fragmentation D) File system caching

Answer: B — It's a classic illustration of resource allocation issues, where philosophers (processes) compete for shared forks (resources), risking deadlock if not handled carefully.

Q48. What are the four necessary conditions for deadlock (Coffman conditions)? A) Mutual exclusion, Hold and wait, No preemption, Circular wait B) Starvation, Aging, Preemption, Priority C) Locking, Blocking, Waiting, Signaling D) Fragmentation, Paging, Swapping, Thrashing

Answer: A — All four conditions must hold simultaneously for a deadlock to occur; breaking any one prevents deadlock.

Q49. What is "Hold and Wait" in deadlock conditions? A) A process holds at least one resource while waiting to acquire additional resources held by others B) A process holds no resources C) A process waits indefinitely without holding anything D) None of the above

Answer: A — This condition means processes don't release held resources while waiting for others, contributing to potential deadlock.

Q50. What is "Circular Wait" in deadlock? A) A set of processes waiting for each other in a circular chain B) A process waiting for itself C) No waiting occurs D) Processes waiting in a linear sequence with no cycle

Answer: A — Circular wait means P1 waits for a resource held by P2, P2 waits for P3, ..., and Pn waits for P1, forming a cycle.

Q51. What is deadlock prevention? A) Ensuring at least one of the four necessary conditions never holds B) Detecting deadlock after it occurs C) Ignoring deadlock entirely D) Allowing deadlock to resolve itself

Answer: A — Prevention works by structurally eliminating one of mutual exclusion, hold-and-wait, no-preemption, or circular wait.

Q52. What is deadlock avoidance and which algorithm is commonly used? A) Banker's Algorithm — dynamically checks if resource allocation leaves the system in a safe state B) Round Robin C) FCFS D) Aging

Answer: A — The Banker's Algorithm simulates resource allocation to ensure the system never enters an unsafe state that could lead to deadlock.

Q53. What is a "safe state" in the context of the Banker's Algorithm? A) A state where the system can allocate resources to each process in some order without deadlock B) A state with no processes C) A state where deadlock has occurred D) A state where all resources are free

Answer: A — In a safe state, there exists at least one safe sequence in which all processes can complete without deadlock.

Q54. What is deadlock detection and recovery? A) Allowing deadlock to occur, then detecting it (e.g., via resource-allocation graph/wait-for graph) and recovering (e.g., by killing/preempting processes) B) Preventing deadlock from ever occurring C) Ignoring deadlock permanently D) Avoiding deadlock before allocation

Answer: A — Unlike prevention/avoidance, this approach allows deadlock, periodically checks for it, and resolves it via process termination or resource preemption.

Q55. What is the "Ostrich Algorithm" for deadlock? A) Ignore the problem — assume deadlocks are rare and just reboot if it happens B) An advanced prevention algorithm C) A detection algorithm D) A recovery algorithm using rollback

Answer: A — Used by many general-purpose OSes (like older UNIX) since deadlocks are rare enough that prevention/avoidance overhead isn't worth it.

Q56. What is a race condition? A) When multiple processes/threads access shared data concurrently and the outcome depends on timing/order of execution B) A CPU speed comparison C) A type of scheduling D) A memory leak

Answer: A — Race conditions lead to unpredictable/incorrect results and are prevented using synchronization mechanisms like locks or semaphores.

Q57. What is a monitor in synchronization? A) A hardware display device B) A high-level synchronization construct that encapsulates shared data and the procedures that operate on it, ensuring mutual exclusion automatically C) A scheduling algorithm D) A type of interrupt

Answer: B — Monitors provide condition variables (wait/signal) and automatically enforce mutual exclusion, simplifying concurrent programming compared to raw semaphores.

Q58. What is priority inversion? A) A high-priority process is indirectly blocked by a low-priority process holding a needed resource B) A low priority process runs before high priority always C) Priorities are reversed by the scheduler intentionally D) A deadlock type

Answer: A — This happens when a low-priority process holds a lock needed by a high-priority process, while a medium-priority process preempts the low-priority one, delaying the high-priority task. Solved via priority inheritance.

Q59. What is "priority inheritance" used to solve? A) Priority inversion B) Deadlock C) Starvation only D) Thrashing

Answer: A — The low-priority process temporarily inherits the higher priority of the process waiting on its held resource, so it can finish and release the resource sooner.

Q60. Which of these is NOT a way to break a deadlock condition? A) Eliminating circular wait via resource ordering B) Allowing preemption of resources C) Guaranteeing mutual exclusion for all resources D) Requiring processes to request all resources at once (eliminating hold and wait)

Answer: C — Enforcing mutual exclusion for all resources doesn't prevent deadlock — in fact mutual exclusion is one of the necessary conditions FOR deadlock, so guaranteeing it everywhere doesn't help break it. (Some resources are inherently non-shareable, so mutual exclusion often can't be eliminated at all — the other three conditions are the usual targets.)

Section 4: Memory Management & Virtual Memory (Q61–Q80)

Q61. What is paging? A) A memory management scheme that eliminates need for contiguous allocation by dividing memory into fixed-size blocks B) A scheduling technique C) A file compression method D) A type of deadlock

Answer: A — Paging divides logical memory into pages and physical memory into frames of the same size, mapped via a page table, avoiding external fragmentation.

Q62. What is segmentation? A) Memory management scheme that divides memory into variable-sized segments based on logical divisions (e.g., code, stack, heap) B) Same as paging C) A CPU scheduling method D) A disk formatting technique

Answer: A — Unlike paging's fixed-size blocks, segmentation reflects the logical structure of a program, though it can cause external fragmentation.

Q63. What is the difference between internal and external fragmentation? A) Internal fragmentation is wasted space within an allocated block; external fragmentation is wasted space between allocated blocks B) They are the same C) Internal fragmentation only occurs in segmentation D) External fragmentation only occurs in paging

Answer: A — Internal fragmentation occurs when allocated memory is larger than requested (common in fixed-size paging); external fragmentation occurs when free memory is split into small non-contiguous blocks (common in segmentation/dynamic partitioning).

Q64. What is a page fault? A) An error when a program accesses a page not currently in physical memory B) A hardware failure C) A type of deadlock D) A CPU scheduling event

Answer: A — On a page fault, the OS must fetch the required page from disk (secondary storage) into RAM, which involves the page replacement mechanism if memory is full.

Q65. What is thrashing? A) A state where the system spends more time swapping pages than executing processes, due to excessive paging B) A type of deadlock C) A CPU scheduling algorithm D) A disk formatting error

Answer: A — Thrashing occurs when the degree of multiprogramming is too high, causing constant page faults and severely degrading performance.

Q66. Which page replacement algorithm replaces the page that hasn't been used for the longest time? A) FIFO B) LRU (Least Recently Used) C) Optimal D) Random

Answer: B — LRU approximates future behavior based on past usage, assuming pages used recently will likely be used again soon.

Q67. Which page replacement algorithm is theoretically optimal but not implementable in practice? A) FIFO B) LRU C) Optimal (Belady's algorithm) — replaces the page that won't be used for the longest time in the future D) Second Chance

Answer: C — The Optimal algorithm requires future knowledge of page references, making it a theoretical benchmark rather than a practical implementation.

Q68. What is "Belady's Anomaly"? A) A phenomenon where increasing the number of page frames results in MORE page faults (can occur with FIFO) B) A type of deadlock C) A CPU scheduling issue D) A memory leak pattern

Answer: A — This counterintuitive behavior can happen with FIFO page replacement, though algorithms like LRU are not susceptible to it (they follow the stack property).

Q69. What is demand paging? A) Pages are loaded into memory only when they are needed/referenced, not all at once B) All pages are loaded at process start C) A type of segmentation D) A CPU scheduling technique

Answer: A — Demand paging reduces memory usage and speeds up process startup by loading pages lazily, triggering page faults when needed.

Q70. What is the purpose of a Translation Lookaside Buffer (TLB)? A) A cache that speeds up virtual-to-physical address translation by storing recent page table entries B) A type of hard disk C) A CPU scheduling table D) A deadlock detection tool

Answer: A — Without a TLB, every memory access would require an extra memory lookup into the page table, doubling access time; the TLB caches frequent translations for speed.

Q71. What is virtual memory? A) A technique that allows execution of processes not completely loaded in physical memory, giving the illusion of a larger memory space B) Physical RAM only C) A type of cache memory D) A CPU register

Answer: A — Virtual memory uses techniques like paging/segmentation combined with secondary storage to let programs use more memory than physically available.

Q72. What is a "dirty bit" in paging? A) A flag indicating whether a page has been modified since it was loaded into memory B) A flag indicating a page is corrupted C) A CPU error flag D) A network flag

Answer: A — If the dirty bit is set, the page must be written back to disk before being replaced; if clean (unmodified), it can simply be discarded (already matches disk copy).

Q73. What is swapping? A) Moving an entire process between main memory and secondary storage (disk) B) Swapping two CPU registers C) A type of page replacement algorithm D) A file system operation

Answer: A — Swapping temporarily moves whole processes out to disk to free up memory, used in memory-constrained systems, distinct from paging which moves individual pages.

Q74. Which memory allocation strategy fits the smallest available block that's still big enough for the request? A) First Fit B) Best Fit C) Worst Fit D) Next Fit

Answer: B — Best Fit minimizes leftover space per allocation but can create many small unusable fragments over time and is slower (requires scanning all blocks).

Q75. Which strategy allocates the first block found that's large enough? A) First Fit B) Best Fit C) Worst Fit D) None

Answer: A — First Fit is generally fast since it stops searching as soon as a suitable block is found, though it can leave small fragments near the start of memory.

Q76. Which strategy allocates the largest available block, leaving the biggest leftover free space? A) First Fit B) Best Fit C) Worst Fit D) Next Fit

Answer: C — Worst Fit aims to leave a usably large leftover fragment, though it can waste large blocks over time and is not commonly preferred.

Q77. What is a page table used for? A) Mapping logical/virtual page numbers to physical frame numbers B) Storing process priorities C) Storing file metadata D) Scheduling CPU time

Answer: A — Each process has its own page table maintained by the OS, used by the MMU (Memory Management Unit) to translate addresses during execution.

Q78. What is a multi-level page table used for? A) To reduce memory overhead of storing a huge single-level page table by hierarchically dividing it B) To speed up CPU scheduling C) To manage disk partitions D) To handle deadlocks

Answer: A — For systems with large address spaces, a single-level page table would be enormous; multi-level (hierarchical) page tables save space by only allocating tables for used regions.

Q79. What does "working set" refer to in memory management? A) The set of pages a process is actively using/referencing in a given time window B) All pages ever used by a process C) The set of pages currently on disk D) The total physical memory size

Answer: A — The working set model helps the OS determine how many frames a process needs to avoid thrashing, based on its recent locality of reference.

Q80. What is the difference between logical (virtual) address and physical address? A) Logical address is generated by the CPU during program execution; physical address is the actual location in memory hardware B) They are always identical C) Physical address is generated by the compiler D) Logical address is used only for I/O

Answer: A — The MMU translates logical addresses (used by the program) into physical addresses (actual RAM locations) at runtime.

Section 5: File Systems, I/O & Disk Scheduling (Q81–Q100)

Q81. What is a file system? A) A method the OS uses to organize, store, retrieve, and manage data on storage devices B) A CPU scheduling technique C) A type of RAM D) A network protocol

Answer: A — Examples include NTFS, ext4, FAT32, each managing how files/directories are structured, named, and accessed on disk.

Q82. What is an inode (in Unix/Linux file systems)? A) A data structure storing metadata about a file (permissions, owner, size, pointers to data blocks) but not the filename B) The actual file content C) A type of directory D) A CPU register

Answer: A — The filename is stored separately in the directory entry, which points to the inode containing the file's actual metadata and data block pointers.

Q83. What is the difference between hard link and soft (symbolic) link? A) A hard link points directly to the inode; a soft link is a separate file containing a path to the target file B) They are identical C) Soft links cannot cross file systems while hard links can, in all cases D) Hard links can link to directories freely on all systems

Answer: A — Hard links share the same inode as the original file (deleting original doesn't remove data until all links are gone); soft links break if the target is moved/deleted since they just store a path.

Q84. Which disk scheduling algorithm serves requests in the order they arrive? A) FCFS B) SCAN C) SSTF D) C-SCAN

Answer: A — FCFS disk scheduling is simple and fair but doesn't optimize for seek time/head movement.

Q85. What is SSTF (Shortest Seek Time First) disk scheduling? A) Services the request closest to the current head position next B) Services requests in FIFO order C) Services requests in a circular pattern only D) Ignores head position

Answer: A — SSTF minimizes seek time for each individual request but can cause starvation of requests far from the frequently-visited areas.

Q86. What is the SCAN (elevator) disk scheduling algorithm? A) The disk arm moves in one direction, servicing requests, then reverses direction at the end, like an elevator B) Random access to any request C) Only moves in one fixed direction and stops D) Same as FCFS

Answer: A — SCAN reduces the variance in wait times compared to SSTF, avoiding starvation by continuing sweeps back and forth across the disk.

Q87. What is C-SCAN (Circular SCAN)? A) Disk arm moves in one direction servicing requests, then jumps back to the beginning without servicing on the return trip B) Same as SCAN exactly C) Services requests randomly D) A memory scheduling technique

Answer: A — C-SCAN provides more uniform wait times by treating the disk as circular, avoiding the bias SCAN has toward middle cylinders.

Q88. What is disk fragmentation? A) Files being stored in non-contiguous blocks scattered across the disk B) A type of RAM error C) A CPU scheduling delay D) A network latency issue

Answer: A — Fragmentation slows access time since the disk head must move more to read scattered file blocks; defragmentation reorganizes data to improve performance (mainly relevant to HDDs, not SSDs).

Q89. What is journaling in file systems? A) A technique that logs changes before committing them, to allow recovery after crashes B) A type of page replacement C) A CPU scheduling method D) A network file transfer method

Answer: A — Journaling file systems (like ext3/ext4, NTFS) keep a log of pending changes so the system can recover to a consistent state after a crash or power failure.

Q90. What is RAID and why is it used? A) Redundant Array of Independent Disks — used for data redundancy and/or performance improvement by combining multiple disks B) A CPU cache type C) A scheduling algorithm D) A file compression method

Answer: A — Different RAID levels (0, 1, 5, 6, 10, etc.) offer different tradeoffs between speed, redundancy/fault tolerance, and storage efficiency.

Q91. What is the difference between RAID 0 and RAID 1? A) RAID 0 stripes data for performance (no redundancy); RAID 1 mirrors data for redundancy B) They are the same C) RAID 1 stripes and RAID 0 mirrors D) Neither offers any benefit

Answer: A — RAID 0 improves speed but offers zero fault tolerance (losing one disk loses all data); RAID 1 duplicates data across disks for fault tolerance at the cost of storage efficiency.

Q92. What is buffering in I/O management? A) Temporarily storing data in memory to accommodate differences in speed between devices/processes B) A type of scheduling algorithm C) A file naming convention D) A deadlock resolution method

Answer: A — Buffering helps smooth out speed mismatches (e.g., between a fast CPU and slower disk/network) and can allow overlap of computation with I/O.

Q93. What is spooling? A) Using disk as a buffer for devices like printers, allowing multiple jobs to queue without waiting for the device to be free B) A CPU scheduling method C) A memory paging technique D) A network security protocol

Answer: A — SPOOL (Simultaneous Peripheral Operations On-Line) lets processes continue without blocking while a shared device (like a printer) processes queued jobs sequentially.

Q94. What is the difference between blocking and non-blocking I/O? A) Blocking I/O halts the calling process until the operation completes; non-blocking I/O returns immediately, letting the process continue and check status later B) They are identical C) Non-blocking I/O never completes D) Blocking I/O is always faster

Answer: A — Non-blocking (and asynchronous) I/O improves responsiveness and concurrency, especially important in servers handling many connections.

Q95. What is DMA (Direct Memory Access)? A) A technique allowing I/O devices to transfer data directly to/from memory without CPU intervention for each byte B) A type of virtual memory C) A CPU scheduling algorithm D) A file system type

Answer: A — DMA frees the CPU from managing every data transfer, greatly improving efficiency for large data transfers (e.g., disk-to-memory).

Q96. What is the boot process / bootstrapping in an OS? A) The process of loading the OS kernel into memory and starting it when a computer powers on B) A file compression technique C) A CPU scheduling method D) A memory allocation strategy

Answer: A — Typically involves BIOS/UEFI firmware locating and loading a bootloader, which then loads the OS kernel into memory to begin execution.

Q97. What is the difference between a monolithic kernel and a microkernel? A) Monolithic kernel runs most OS services in kernel space; microkernel keeps the kernel minimal, running most services in user space B) They are the same C) Microkernels are always faster in every case D) Monolithic kernels have no drivers

Answer: A — Monolithic kernels (e.g., traditional Linux) offer speed due to fewer context switches; microkernels (e.g., Minix, QNX) offer better modularity/stability since a crashing service doesn't crash the whole kernel.

Q98. What is a device driver? A) Software that allows the OS to communicate with and control a specific hardware device B) A type of file system C) A CPU scheduling algorithm D) A network protocol

Answer: A — Device drivers act as a translator between generic OS I/O calls and device-specific hardware commands.

Q99. What is the purpose of an OS's "shell"? A) A user interface (CLI or GUI) that allows users to interact with the OS by issuing commands B) A part of the CPU C) A memory management module D) A file compression tool

Answer: A — Examples include Bash, PowerShell, and various GUI shells — they interpret user commands and pass requests to the kernel via system calls.

Q100. What is the primary difference between a real-time OS (RTOS) and a general-purpose OS? A) RTOS guarantees processing within strict, predictable time constraints (deterministic response); general-purpose OS prioritizes overall throughput/fairness without hard deadlines B) RTOS is always slower C) General-purpose OS is used only in embedded systems D) There is no meaningful difference

Answer: A — RTOS (e.g., used in embedded/industrial control systems) is designed for predictability and meeting deadlines (hard or soft real-time), unlike general-purpose OSes like Windows/Linux desktop editions which optimize for average-case performance.

Quick Revision Tips for Welldev OS Round

- Be ready to **explain, not just recall** — interviewers often ask "why" after the MCQ answer (e.g., why does SJF minimize average waiting time).
- Practice **numerical problems**: Gantt charts for FCFS/SJF/SRTF/RR/Priority scheduling, and page replacement traces for FIFO/LRU/Optimal.
- Know **real-world tie-ins**: Linux process states, `fork()`/`exec()` behavior, `nice` values, common deadlock scenarios in multithreaded code.
- Review the **Dining Philosophers** and **Producer-Consumer** solutions using semaphores — these are very commonly asked to code/explain live.
- Understand **paging vs segmentation** and **internal vs external fragmentation** clearly — a frequent "compare and contrast" question.